CYLIXLAB · STUDY GUIDE

# Module 02

## Deep Learning & Neural Network Architectures

Theory | MLP · Backpropagation · Activations · CNNs · RNNs · Loss Functions

### What you will master

| Section | Topic | You should be able to… |
|---|---|---|
| 1 | Multi-Layer Perceptron (MLP) | Describe neurons and layers, run a forward pass and count parameters |
| 2 | Backpropagation | Apply the chain rule to compute gradients and explain vanishing / exploding gradients |
| 3 | Activation Functions | Pick the right activation for hidden and output layers and justify it |
| 4 | Convolutional Networks (CNN) | Compute output sizes and parameter counts; explain pooling, stride, padding and ResNets |
| 5 | Recurrent Networks (RNN, LSTM, GU) | Explain sequence modelling, BPTT and how gates fix vanishing gradients |
| 6 | Loss Functions | Match task, output activation and loss; compute cross-entropy by hand |
| 7 | Revision Kit | Use the formula sheet, training checklist and pitfall list before the MCQs |

**KEY IDEA**

**Deep learning in one sentence:** stack many differentiable layers, measure the error with a **loss function**, use **backpropagation** to get every gradient efficiently, and update the weights with an optimizer such as Adam (Module 01). The architectures in this module (MLP, CNN, RNN) differ only in *how the layers are wired* to exploit the structure of the data.

| Data type | Typical architecture | Built-in assumption (inductive bias) |
|---|---|---|
| Tabular features | MLP | None: all features may interact |
| Images, grids | CNN | Local patterns; the same pattern can appear anywhere (translation equivariance) |
| Sequences, time series, text | RNN / LSTM / GRU (and Transformers) | Order matters; the same rule applies at every time step |

## 1 Multi-Layer Perceptron (MLP)

The **perceptron** is a single artificial neuron: it takes a weighted sum of its inputs plus a bias and passes it through an **activation function**. An **MLP** (also called a feed-forward or fully-connected network) stacks layers of such neurons so that information flows from input to output with no loops.

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad a = f(z)$$

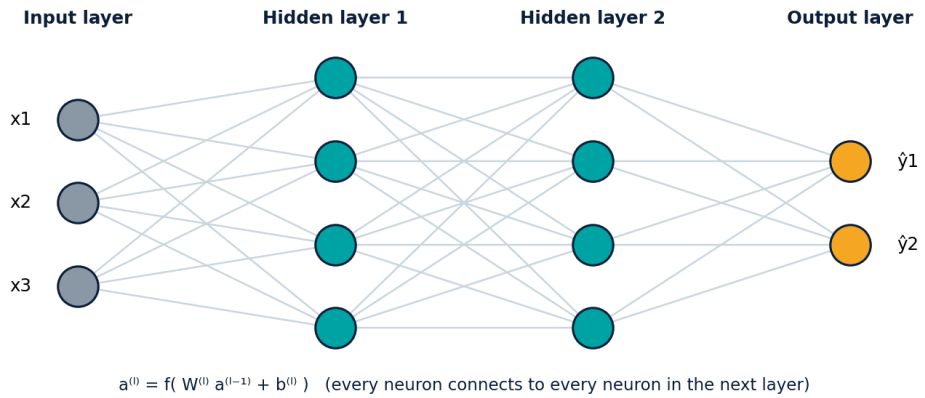



Figure 1: A 3–4–4–2 MLP. Every neuron feeds every neuron in the next layer (“dense” layers).

## 1.1 Forward pass

For layer l with weight matrix $W^{(l)}$ (shape: neurons in layer l × neurons in layer l−1), bias vector $b^{(l)}$ and activation f:

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}), \qquad \mathbf{a}^{(0)} = \mathbf{x}$$

The last layer’s output $\hat{y} = a^{(L)}$ is compared with the target y by a loss function. Training adjusts all W and b to reduce that loss.

## 1.2 Why non-linearity and depth matter

- Without a non-linear activation, any stack of layers collapses into **one linear transformation** ($W_2(W_1x) = (W_2W_1)x$). Depth would add nothing.
- The **Universal Approximation Theorem**: an MLP with one sufficiently wide hidden layer and a non-linear activation can approximate any continuous function on a compact domain. It says a solution *exists*, not that it is easy to *learn*.
- In practice **deeper** networks learn hierarchical features (simple → complex) with far fewer neurons than a single enormous layer.

## 1.3 Counting parameters

A dense layer with $n_{in}$ inputs and $n_{out}$ neurons has $\mathbf{n_{in} \cdot n_{out}}$ **weights +** $\mathbf{n_{out}}$ **biases**.

**WORKED EXAMPLE**
Network 784 → 128 → 64 → 10 (e.g. MNIST digits):
Layer 1: 784·128 + 128 = **100,480**. Layer 2: 128·64 + 64 = **8,256**. Layer 3: 64·10 + 10 = **650**.
Total = **109,386** trainable parameters.

| Hyper-parameter | Typical choice | Notes |
|---|---|---|
| Hidden layers / width | 1–5 layers; 64–1024 units | Start small, grow until the validation error stops improving |
| Hidden activation | ReLU (or GELU) | See Section 3 |
| Output layer | Linear / Sigmoid / Softmax | Decided by the task (Section 6) |
| Optimizer | Adam / AdamW, $\eta \approx 10^{-3}$ | From Module 01 |
| Regularization | Weight decay, dropout, early stopping | Large MLPs overfit easily |

# 2 Backpropagation

Training needs the gradient of the loss with respect to *every* weight, which can be millions. **Backpropagation** computes all of them in one backward sweep by applying the **chain rule** layer by layer, reusing intermediate results. It costs about the same as one forward pass. Modern libraries automate it with **automatic differentiation** on a computational graph.

## 2.1 The chain rule on a graph

If L depends on w through a → z → w, the gradient is the product of the local derivatives along that path:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

## 2.2 The algorithm

- **Forward pass:** compute and *store* all $z^{(l)}$ and $a^{(l)}$, then the loss L.
- **Output error:** $\delta^{(L)} = \partial L/\partial z^{(L)}$. For softmax + cross-entropy (or sigmoid + BCE) this simplifies to $\hat{\mathbf{y}} - \mathbf{y}$.
- **Propagate backward:** pass the error through the transposed weights and multiply by the local activation derivative.
- **Gradients:** $\partial L/\partial W^{(l)}$ uses the error times the layer's *input* activations; $\partial L/\partial b^{(l)}$ equals the error.
- **Update:** $\theta \leftarrow \theta - \eta \nabla L$ (or an Adam step).

$$\boldsymbol{\delta}^{(l)} = (W^{(l+1)\top} \boldsymbol{\delta}^{(l+1)}) \odot f'(\mathbf{z}^{(l)})$$

$$\frac{\partial L}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\top}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

**WORKED EXAMPLE**
One sigmoid neuron: x = 2, w = 0.5, b = 0, target y = 1, loss $L = \tfrac{1}{2}(a - y)^2$.
**Forward:** $z = 0.5 \cdot 2 = 1$; $a = \sigma(1) = 0.731$; $L = \tfrac{1}{2}(-0.269)^2 \approx 0.036$.
**Backward:** $\partial L/\partial a = a - y = -0.269$; $\partial a/\partial z = a(1 - a) = 0.197$; $\partial z/\partial w = x = 2$.
$\partial L/\partial w = (-0.269)(0.197)(2) \approx \mathbf{-0.106}$. With $\eta = 0.5$: $w \leftarrow 0.5 - 0.5(-0.106) = \mathbf{0.553}$. The weight increases, which pushes a closer to 1.

## 2.3 Vanishing and exploding gradients

The gradient reaching an early layer is a **product of many factors** (weights × activation derivatives). If these factors are typically below 1 the product shrinks exponentially, so early layers stop learning (**vanishing**). If they are above 1 it blows up (**exploding**). Sigmoid's derivative is at most 0.25, so ten sigmoid layers can shrink a gradient by about $0.25^{10} \approx 10^{-6}$.

| Problem | Symptoms | Remedies |
|---|---|---|
| Vanishing gradients | Early layers barely change; loss plateaus | ReLU-family activations, He/Xavier initialization, batch normalization, residual (skip) connections, LSTM/GRU gates |
| Exploding gradients | Loss spikes or becomes NaN; huge weights | Gradient clipping, lower learning rate, proper initialization, normalization |
| Dead ReLUs | Neurons output 0 for all inputs | Lower learning rate, Leaky ReLU / GELU, better initialization |

**Weight initialization.** Weights must not all be equal (symmetry) and should keep activation variance stable across layers. **Xavier/Glorot** (variance ≈ $2/(n_{in}+n_{out})$) suits tanh/sigmoid; **He** (variance ≈ $2/n_{in}$)

suits ReLU.

**EXAM TIP**
Numerical **gradient checking** compares backprop gradients with $(L(\theta+\varepsilon) - L(\theta-\varepsilon))/2\varepsilon$. It is slow, so use it only to debug a hand-written implementation.

## 3 Activation Functions

An activation function injects the non-linearity that gives networks their expressive power. Good activations are non-linear, differentiable (almost everywhere), cheap to compute, and keep gradients healthy.

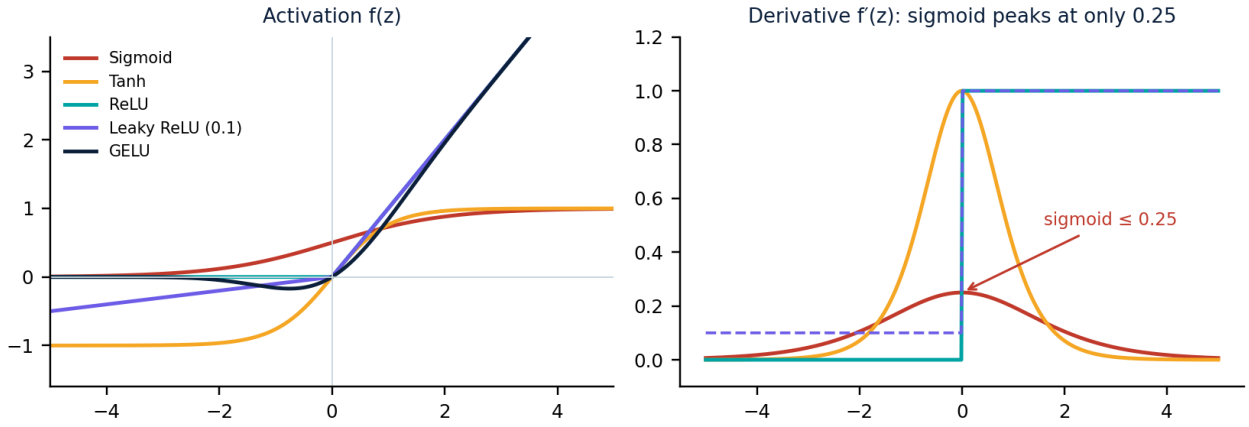



Figure 2: Common activations (left) and their derivatives (right).

| Function | Formula | Range | Pros | Cons / notes |
|---|---|---|---|---|
| Sigmoid | $1/(1+e^{-z})$ | (0, 1) | Probability output; smooth | Saturates → vanishing gradient; not zero-centred |
| Tanh | $(e^{z}-e^{-z})/(e^{z}+e^{-z})$ | (−1, 1) | Zero-centred; steeper than sigmoid | Still saturates at the extremes |
| **ReLU** | max(0, z) | [0, ∞) | Fast, no saturation for z>0; sparse activations; default for hidden layers | “Dying ReLU”: neurons stuck at 0 |
| Leaky ReLU | z if z>0 else αz (α≈0.01) | (−∞, ∞) | Keeps a small gradient for z<0 | Extra hyper-parameter α |
| GELU / SiLU | $z\cdot\Phi(z)$ / $z\cdot\sigma(z)$ | ≈[−0.17, ∞) | Smooth; standard in Transformers | Slightly costlier |
| **Softmax** | $e^{z_i} / \Sigma_j e^{z_j}$ | (0, 1), sums to 1 | Multi-class probabilities | Output layer only |

### Derivatives you should know

- Sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, maximum **0.25** at z = 0.
- Tanh: $\tanh'(z) = 1 - \tanh^2(z)$, maximum **1** at z = 0.
- ReLU: 1 for z > 0, 0 for z < 0 (undefined at 0; frameworks use 0).

**WORKED EXAMPLE**
Softmax for logits z = [2.0, 1.0, 0.1]: $e^z$ = [7.389, 2.718, 1.105], sum = 11.212, so probabilities = **[0.659, 0.242, 0.099]**. They sum to 1; the largest logit gets the largest probability. Subtracting max(z) before exponentiating gives identical results and avoids overflow.

| Where | Recommended activation |
|---|---|
| Hidden layers (MLP / CNN) | ReLU by default; Leaky ReLU or GELU if neurons die or in Transformers |
| Hidden layers (RNN cell) | tanh (state), sigmoid (LSTM/GRU gates) |
| Output: regression | Linear (no activation) |
| Output: binary / multi-label | Sigmoid (one per label) |
| Output: multi-class (exactly one class) | Softmax |

**COMMON PITFALL**
Do not put a softmax or sigmoid in the *hidden* layers of a deep network just because it outputs probabilities. Saturation will stall learning. Also, many frameworks expect **raw logits** and apply softmax/sigmoid inside the loss for stability.

# 4 Convolutional Neural Networks (CNNs)

Flattening an image into an MLP ignores its 2-D structure and needs huge numbers of weights (a 224×224×3 image has 150,528 inputs). CNNs exploit two ideas: **local connectivity** (each neuron looks at a small patch) and **weight sharing** (the same small filter slides across the whole image). This gives far fewer parameters and detects a pattern wherever it appears.

## 4.1 The convolution operation

A **kernel** (filter) is a small weight grid. At each position, multiply it element-wise with the underlying patch and sum the results to produce one value of the **feature map** (deep-learning "convolution" is technically cross-correlation). Different kernels detect different features: edges, textures, shapes.

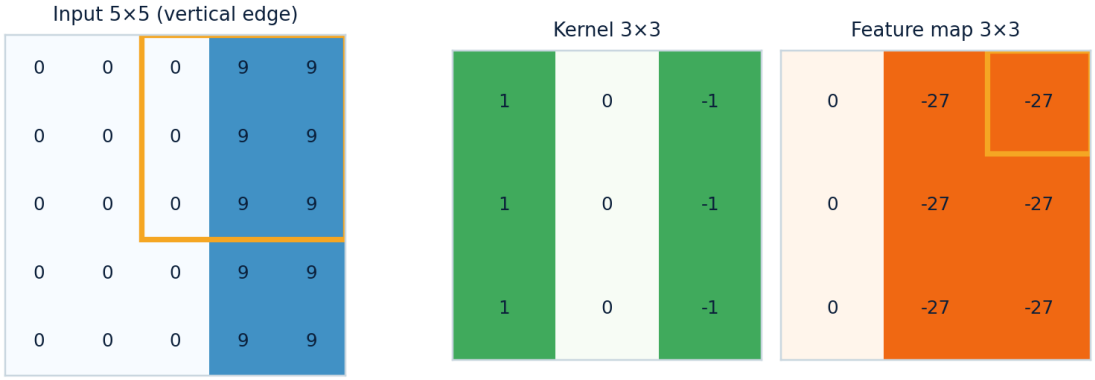



Figure 3: A vertical-edge kernel on a 5×5 image. The highlighted 3×3 window produces the highlighted output (output −27: a strong response at the edge; flat regions give 0).

## 4.2 Stride, padding and output size

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

W = input size, K = kernel size, P = padding, S = stride. **Padding** (e.g. P = 1 for a 3×3 kernel) preserves spatial size ("same"); no padding is "valid". **Stride** > 1 downsamples.

$$\text{Params} = (K \times K \times C_{in} + 1) \times C_{out}$$

**WORKED EXAMPLE**
Input 32×32×3, 16 filters of 5×5, stride 1, no padding: O = (32 − 5)/1 + 1 = **28**, so the output is 28×28×16.
Parameters = (5·5·3 + 1)·16 = **1,216**.
Input 224, 3×3 kernel, stride 2, padding 1: O = ⌊(224 − 3 + 2)/2⌋ + 1 = 111 + 1 = **112**.

## 4.3 Pooling and the CNN pipeline

**Max pooling** (e.g. 2×2, stride 2) keeps the strongest response in each window; **average pooling** averages it. Pooling shrinks the feature map, adds small translation invariance and has **no trainable parameters**. **Global average pooling** replaces large dense layers at the end of modern CNNs.

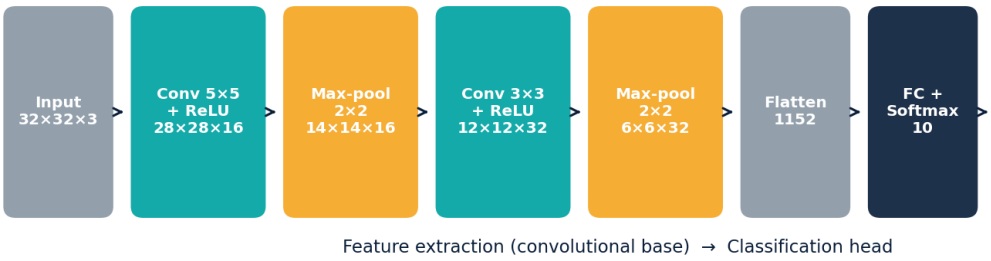



Figure 4: A small CNN for 10-class image classification (feature maps shown as height × width × channels).

- **Hierarchy of features:** early layers learn edges and colours, middle layers textures and parts, deep layers whole objects.
- **Receptive field:** the region of the input a neuron “sees” grows with depth; stacking two 3×3 convolutions covers 5×5 with fewer parameters than one 5×5.
- **1×1 convolutions** mix channels cheaply (dimension reduction).
- **Batch normalization** standardizes each layer’s activations per mini-batch, stabilizing and speeding up training. **Data augmentation** (flips, crops, colour jitter) fights overfitting.
- **Transfer learning:** reuse a convolutional base pre-trained on a large dataset (e.g. ImageNet) and fine-tune the head. This is the best default when labelled data is scarce.

| Architecture | Key idea | Why it matters |
|---|---|---|
| LeNet-5 (1998) | Conv + pool + dense | First practical CNN (digits) |
| AlexNet (2012) | Deeper CNN, ReLU, dropout, GPUs | Started the deep-learning boom on ImageNet |
| VGG | Stacks of small 3×3 convolutions | Simple, uniform design |
| Inception / GoogLeNet | Parallel filters of different sizes | Efficiency and multi-scale features |
| **ResNet** (2015) | **Skip connections**: output = F(x) + x | Gradients flow through the identity path, enabling 50–150+ layers |
| EfficientNet / MobileNet | Compound scaling / depthwise-separable convolutions | Accuracy per FLOP; mobile deployment |

# 5 Recurrent Neural Networks (RNN, LSTM, GRU)

MLPs and CNNs treat each input independently and need a fixed size. For **sequences** (text, speech, sensor streams, stock prices) the order and context matter and the length varies. An **RNN** keeps a **hidden state** $h_t$ that summarizes everything seen so far, and applies the *same* weights at every time step.

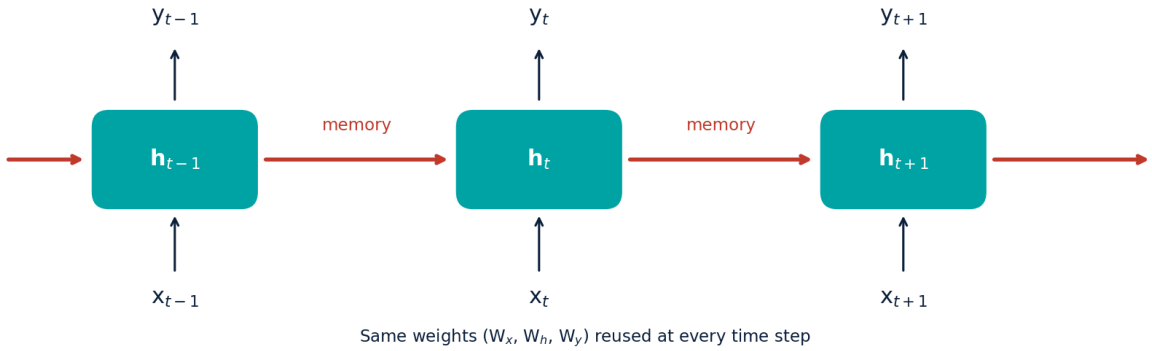



Figure 5: An RNN unrolled through time. The hidden state carries memory forward.

$$h_t = \tanh(W_h\, h_{t-1} + W_x\, x_t + b), \qquad y_t = W_y\, h_t + b_y$$

## 5.1 Backpropagation Through Time (BPTT)

To train, the RNN is **unrolled** over the sequence and ordinary backpropagation is applied; the gradient for a shared weight is the **sum** over all time steps. Because the gradient to an early step is multiplied by $W_h$ and $\tanh'$ many times, it **vanishes** (or explodes) on long sequences, so a plain RNN struggles to remember information from 50+ steps back. **Truncated BPTT** limits how far back gradients flow, and **gradient clipping** tames explosions.

## 5.2 LSTM: gated memory

The **Long Short-Term Memory** cell adds a separate **cell state** $c_t$, a "conveyor belt" modified only by gentle, learned gates (sigmoid outputs between 0 and 1):

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$$

- **Forget gate $f_t$:** how much old memory to keep. **Input gate $i_t$:** how much new information to write. **Output gate $o_t$:** how much memory to expose.
- Because $c_t$ is updated *additively*, gradients can flow across many steps with little decay, which fixes most of the vanishing-gradient problem. (A forget-gate bias initialized near 1 helps early training.)
- **GRU** merges the cell and hidden state and uses two gates (update and reset): fewer parameters, faster, often similar accuracy.

**WORKED EXAMPLE**
LSTM parameter count = 4·(h·(h + d) + h), with four weight sets (f, i, o, candidate). For input size d = 10 and hidden size h = 20: 4·(20·30 + 20) = 4·620 = **2,480** parameters. A GRU has three sets, so 3·620 = 1,860.

### 5.3 Sequence tasks and extensions

| Pattern | Example | How |
|---|---|---|
| Many-to-one | Sentiment of a review | Use the final hidden state |
| One-to-many | Image captioning | Generate one output per step |
| Many-to-many (aligned) | Part-of-speech tagging | Output at every step |
| Many-to-many (seq2seq) | Machine translation | Encoder RNN → context → decoder RNN (+ attention) |

**Bidirectional RNNs** read the sequence forwards and backwards (useful when the whole input is available, e.g. tagging). **Attention** lets the model look directly at any earlier position, and **Transformers** build on attention alone: they process all positions in parallel, scale better and have largely replaced RNNs for language. RNNs remain useful for small, streaming or low-latency problems.

| Model | Long-range memory | Parallel training | Parameters |
|---|---|---|---|
| Vanilla RNN | Poor | No (sequential) | Fewest |
| LSTM | Good | No | 4 gate sets |
| GRU | Good | No | 3 gate sets |
| Transformer | Excellent (attention) | **Yes** | Many; scales with data |

## 6 Loss Functions

The loss turns “how wrong is the model?” into one differentiable number that backpropagation can minimize. It must match the **task** and the **output activation**. (Evaluation metrics such as accuracy or F1 are for reporting; they are usually not differentiable, so the loss is a smooth surrogate.)

### 6.1 Regression losses

| Loss | Formula (per batch) | Behaviour |
|---|---|---|
| MSE (L2) | $(1/n)\ \Sigma\ (y - \hat{y})^2$ | Smooth; punishes big errors heavily; sensitive to outliers |
| MAE (L1) | $(1/n)\ \Sigma\ \lvert y - \hat{y}\rvert$ | Robust to outliers; constant gradient, harder to converge precisely |
| Huber (smooth L1) | $\tfrac{1}{2}\ e^2$ if $\lvert e\rvert \le \delta$, else $\delta(\lvert e\rvert - \tfrac{1}{2}\delta)$ | Quadratic near zero, linear for outliers: best of both |

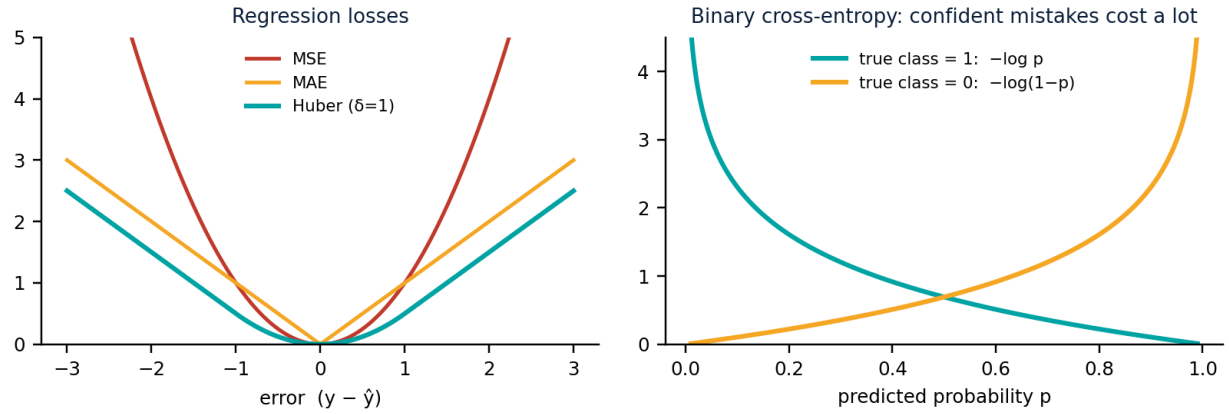



Figure 6: Left: regression losses vs error. Right: cross-entropy grows without bound as a confident prediction becomes wrong.

## 6.2 Classification losses

$$L_{BCE} = -[\, y \log p + (1-y)\log(1-p)\,], \qquad L_{CE} = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

**Binary cross-entropy (BCE)** pairs with a sigmoid output; **categorical cross-entropy (CCE)** pairs with softmax and one-hot targets (with a one-hot target it reduces to −log of the probability given to the *true* class). Cross-entropy equals the negative log-likelihood and measures the KL divergence between true and predicted distributions up to a constant.

**WORKED EXAMPLE**
Softmax output for a 3-class problem is [0.659, 0.242, 0.099] and the true class is the first: L = −ln(0.659) = **0.417**. If the model had given the true class only 0.099, L = −ln(0.099) = **2.31**.
Binary case, y = 1: p = 0.9 gives L = 0.105; p = 0.1 gives L = **2.303** (confident mistakes cost far more).

| Loss | Use when | Note |
|---|---|---|
| Hinge | Max-margin classifiers (SVM) | Zero loss once the margin is satisfied |
| Weighted / focal cross-entropy | Imbalanced classes | Weights or $(1-p)^{\gamma}$ down-weight easy examples |
| Label smoothing | Over-confident networks | Targets like 0.9 / 0.1 instead of 1 / 0 |
| KL divergence | Distillation, VAEs | Distance between two distributions |
| Contrastive / triplet | Embeddings, similarity | Pull similar pairs together, push others apart |

## 6.3 Choosing task, output and loss together

| Task | Output layer | Loss | Target format |
|---|---|---|---|
| Regression | Linear | MSE / MAE / Huber | Real number(s) |
| Binary classification | 1 unit + Sigmoid | BCE | 0 or 1 |
| Multi-class (one label) | K units + Softmax | Categorical CE | One-hot or class index |
| Multi-label (several labels) | K units + K Sigmoids | BCE per label | Multi-hot vector |

**COMMON PITFALL**
**Don’t use MSE for classification** with sigmoid/softmax outputs: gradients become tiny when the model is confidently wrong. Cross-entropy with softmax/sigmoid gives the clean gradient (ŷ − y). Also, never apply softmax twice (once in the model and again inside a loss that expects logits).

# 7 Revision Kit

## 7.1 Regularization and training checklist

| Technique | What it does | Where |
|---|---|---|
| Weight decay (L2) | Penalizes large weights | All networks (use AdamW) |
| Dropout (p ≈ 0.1–0.5) | Randomly zeroes units during training only | Dense layers; some RNN connections |
| Batch / Layer norm | Normalizes activations, stabilizes gradients | CNNs (batch norm), RNNs and Transformers (layer norm) |
| Early stopping | Stop when validation loss stops improving | Always |
| Data augmentation | Creates label-preserving variants | Images, audio, text |
| LR schedule / warm-up | Reduces step size over time | Always worth trying |

## 7.2 One-page formula sheet

| Concept | Formula / rule |
|---|---|
| Neuron | $a = f(w^\top x + b)$ |
| Dense layer params | $n_{in} \cdot n_{out} + n_{out}$ |
| Backprop error | $\delta^{(l)} = (W^{(l+1)^\top} \delta^{(l+1)}) \odot f'(z^{(l)})$; $\partial L/\partial W = \delta\, a^\top$ |
| Softmax + CE gradient | $\partial L/\partial z = \hat{y} - y$ |
| Sigmoid′ / tanh′ | $\sigma(1-\sigma) \le 0.25$ / $1 - \tanh^2 \le 1$ |
| Conv output size | $\lfloor (W - K + 2P)/S \rfloor + 1$ |
| Conv params | $(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$ |
| RNN step | $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$ |
| LSTM params | $4 \cdot (h(h + d) + h)$ |
| LSTM cell | $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$; $h_t = o_t \odot \tanh(c_t)$ |
| BCE / CCE | $-[y \log p + (1-y) \log(1-p)]$ / $-\Sigma\, y_k \log \hat{y}_k$ |

## 7.3 Classic mistakes

- Using a linear activation in all hidden layers (network collapses to linear regression).
- Putting softmax/sigmoid before a loss that already applies it (double activation).
- Using MSE for classification, or accuracy as a training loss.
- Forgetting that pooling has no parameters, or miscounting conv output size by ignoring padding/stride.
- Initializing all weights to the same value (symmetry never breaks).
- Leaving dropout/batch-norm in training mode at test time.
- Expecting a plain RNN to remember dependencies hundreds of steps long.
- Shuffling time-series or sequence data across the time axis before windowing.
- Not normalizing inputs (large raw features slow or destabilize training).
- Judging a model only by training loss without a validation set.

## 7.4 Self-check before the MCQ assessment

- Can you do a forward and backward pass through one neuron by hand?
- Can you compute a conv layer’s output size and parameter count?
- Can you explain why LSTMs mitigate vanishing gradients and ReLU does too?
- Can you choose output activation and loss for regression, binary, multi-class and multi-label tasks?
- Can you explain why sigmoid in deep hidden stacks causes vanishing gradients?

**KEY IDEA**
Next step: attempt the **Module 02 MCQ Assessment**. For each miss, find the matching section above, redo the worked example by hand, then explain in one sentence why the wrong option was tempting.